

Домашнее задание 14

- 1) Для тестирования буду использовать уже созданный мной скрипт из прошлых заданий `./log.sh`, его содержание и пример выполнения команды `bash -x ./log.sh`. Выводит все этапы работы скрипта:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bat log.sh
File: log.sh
1  #!/bin/bash
2
3  timestamp=$(date +%T %S")
4
5  for file in *.log
6  do
7
8  mv "$file" "${file%.log}_${timestamp}.log"
9
10 done

anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bash -x ./log.sh
++ date +%T %S'
+ timestamp='15:08:04 04'
+ for file in *.log
+ mv 'file1_13:21:44 44_14:54:41 41_15:04:10 10.log' 'file1_13:21:44 44_14:54:41 41_15:04:10 10_15:08:04 04.log'
+ for file in *.log
+ mv 'file_13:21:44 44_14:54:41 41_15:04:10 10.log' 'file_13:21:44 44_14:54:41 41_15:04:10 10_15:08:04 04.log'
```

- 2) Теперь беру скрипт, который описан в первой статье. Добавляю туда `set -x/set +x` для автоматической отладки куска скрипта, который кажется нам, что не отработал или имеет ошибки. Результат:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bat test_script.sh
File: test_script.sh
1  #!/bin/bash
2
3  exec 5> debug_output.txt
4
5  BASH_XTRACEFD="5"
6
7  PS4='$LINENO: '
8
9  set -x
10 var="Привет мир!"
11
12 # печать
13 echo "$var"
14
15 # альтернативный способ печати
16 printf "%s\n" "$var"
17 set +x
18
19 echo "Мой дом - это: $HOME"

anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ ./test_script.sh
./test_script.sh: line 3: exec 5: command not found
./test_script.sh: line 5: BASH_XTRACEFD: 5: invalid value for trace file descriptor
10: var='Привет мир!'
13: echo 'Привет мир!'
Привет мир!
16: printf '%s\n' 'Привет мир!'
Привет мир!
17: set +x
Мой дом - это: /home/anastasia
```

- 3) Переписываю еще один скрипт из статьи для теста. И запускаем с параметром `-n`. Результат в том, что скрипт отработал без ошибок, так как на консоль выводятся только ошибки

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bat test_script1.sh
File: test_script1.sh
1  #!/bin/bash
2
3  var="Hello World"
4  echo "$var"
5  echo "Home catalog is=$HOME"
6  echo "My name is=$USER"

anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bash -n ./test_script1.sh
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$
```

- 4) Беру файл со скриптом из лекции change_pswd_auth.sh, где есть ошибки и запускаю с параметром -n. Итог:

```
trap 'echo "Next command: $BASH_COMMAND"; read' DEBUG
#set -x

timestamp=$(date "+%F %S")
value=$1
file=$(find "/home/anastasia" -type f 2>/dev/null)

if [[ "$value" != "yes" && "$value" != "no" ]]; then
    echo "Введите значение замены"
    exit 1
fi

for change in $file
do

    if grep -q "PasswordAuthentication" "$change"; then

        cp "$change" "${change}.bak_${timestamp}"

        sed -i "s/^PasswordAuthentication.*/PasswordAuthentication $value/" "$change"
        echo "Backup: ${change}.bak_${timestamp}"

    fi

done

#sshd -t

if [ $? -ne 0 ]; then
    echo "Invalid file"
    exit 1

#systemctl restart sshd || systemctl restart ssh

echo "PasswordAuthentication set to $value"

anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bash -n ../change_pswd_auth.sh
../change_pswd_auth.sh: line 39: syntax error: unexpected end of file
```

- 5) Снова беру скрипт из предыдущего пункта и запускаю его с параметром -v, который выведет нам в подробном режиме работу скрипта:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bash -v ../change_pswd_auth.sh
#!/bin/bash

trap 'echo "Next command: $BASH_COMMAND"; read' DEBUG
#set -x

timestamp=$(date "+%F %S")
echo "Next command: $BASH_COMMAND"; read
Next command: timestamp=$(date "+%F %S")

value=$1
echo "Next command: $BASH_COMMAND"; read
Next command: value=$1

file=$(find "/home/anastasia" -type f 2>/dev/null)
echo "Next command: $BASH_COMMAND"; read
Next command: file=$(find "/home/anastasia" -type f 2> /dev/null)

if [[ "$value" != "yes" && "$value" != "no" ]]; then
    echo "Введите значение замены"
    exit 1
fi
echo "Next command: $BASH_COMMAND"; read
Next command: [[ "$value" != "yes" && "$value" != "no" ]]
echo "Next command: $BASH_COMMAND"; read
Next command: echo "Введите значение замены"

Введите значение замены
echo "Next command: $BASH_COMMAND"; read
Next command: exit 1
```

Вторая статья

1. Создаю свой список пользователей, с которыми буду работать дальше:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ sudo vim /var/users
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bat /var/users
```

	File: /var/users
1	anastasia
2	margarita
3	sinya
4	nikita
5	aleksandr
6	konstantin

2. Пришлось в самом начале немного переделать скрипт script_addgroup_addusers.sh, так как синтаксис отличается:

```
#!/bin/bash
group=$1
user=$2

sudo addgroup "$group"

#Sudoers
if [ "$group" = it ] || [ "$group" = security ]; then
if ! grep "$group" /etc/sudoers
then
sudo cp /etc/sudoers{,.bkp}
echo "% group ALL=(ALL) ALL" >> /etc/sudoers
fi
elif [ "$user" = admin ]; then
if ! grep "$user" /etc/sudoers
then
sudo cp /etc/sudoers{,.bkp}
echo "$user ALL=(ALL) ALL" >> /etc/sudoers
fi
fi
mkdir -v /home/anastasia/"$group"
sudo useradd "$user" -g "$group" -b /home/anastasia/"$group"
```

3. Теперь создаем скрипт с циклом for и запускаем. Результат:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ ./for.sh
In this line: anastasia
In this line: margarita
In this line: sinyia
In this line: nikita
In this line: aleksandr
In this line: konstantin
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bat for.sh
```

	File: for.sh
1	#!/bin/bash
2	
3	for line in \$(cat /var/users)
4	do
5	echo In this line: \$line
6	done

4. По статье проходит использование for для работы со строками и разделителями IFS и команды cut -d ' ' -f1. Не совсем понятно откуда берется группа. Для теста добавлю группы в файл с пользаками. Вывод после добавления:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ ./for.sh
Username: anastasia Group: it
Username: margarita Group: security
Username: sonya Group: analitycs
Username: nikita Group: it
Username: aleksandr Group: saas
Username: konstantin Group: analitycs
```

5. Теперь переходим в наш изначальный скрипт из пункта 2 и добавляем туда наш цикл for с разделителями:

```
group=$1
user=$2
file=/var/users

IFS=$'\n'

for line in $(cat $file)
do
    user=$(echo $line | cut -d' ' -f1)
    group=$(echo $line | cut -d' ' -f2)
    echo Username: $user Group: $group
done

sudo addgroup "$group"
```

6. Как мы видим добавилась только последняя группа:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ sudo ./script_addgroup_addusers.sh azeti devops
Username: anastasia Group: it
Username: margarita Group: security
Username: sonya Group: analitycs
Username: nikita Group: it
Username: aleksandr Group: saas
Username: konstantin Group: analitycs
info: Selecting GID from range 1000 to 59999 ...
info: Adding group `analitics' (GID 1001) ...
mkdir: created directory '/home/anastasia/analitics'
```

7. Чтобы не увеличивать громоздкость скрипта и чтобы они не повторялись, в статье предложено использовать функцию. Название create_user(). Чтобы если в дальнейшем понадобится можно было вызывать эту функцию командой create_user. Задается до того, как мы к ней обращаемся:

```
create_user() {

    sudo addgroup "$group" 2>/dev/null

    #Sudoers
    if [ "$group" = it ] || [ "$group" = security ]; then
        if ! grep -Fqw "%${group}" /etc/sudoers
        then
            sudo cp /etc/sudoers{,.bkp}
            echo "%$group ALL=(ALL) ALL" >> /etc/sudoers
        fi

    elif [ "$user" = admin ]; then
        if ! grep -Fqw "%${user}" /etc/sudoers
        then
            sudo cp /etc/sudoers{,.bkp}
            echo "$user ALL=(ALL) ALL" >> /etc/sudoers
        fi
    fi
}
```

8. Далее идет работа с select и case. Для начала проверяется работа с select:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ ./select.sh
1) 1
2) 2
3) 3
#? 1
This is number: 1
#? 2
This is number: 2
#? 3
This is number: 3
#? ^C
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bat select.sh
```

	File: select.sh
1	#!/bin/bash
2	
3	select number in 1 2 3
4	do
5	echo "This is number: \$number"
6	done

9. Пример работы скрипта с case:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ ./case.sh
1
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ bat case.sh
```

	File: case.sh
1	#!/bin/bash
2	
3	number=one
4	
5	case \$number in
6	one) echo 1;;
7	two) echo 2;;
8	*) echo "something wrong" ;;
9	esac

10. Объединяем select с case и добавляем в наш скрипт вместо for:

```
#!/bin/bash

create_user() {

#Sudoers
sudo groupadd "$group" 2>/dev/null

if [ "$group" = it ] || [ "$group" = security ]; then
if ! grep -Fqw "%${group}" /etc/sudoers
then
sudo cp /etc/sudoers{,.bkp}
echo "%$group ALL=(ALL) ALL" >> /etc/sudoers
fi

elif [ "$user" = admin ]; then
if ! grep -Fqw "%${user}" /etc/sudoers
then
sudo cp /etc/sudoers{,.bkp}
echo "$user ALL=(ALL) ALL" >> /etc/sudoers
fi

fi

mkdir -v "/home/anastasia/$group"
useradd "$user" -g "$group" -b "/home/anastasia/$group"
echo "User \"$user\" in group \"$group\" created successfully."
}

select option in "Add user" "Show users" "Exit"
do
case $option in
"Add user")
read -rp "Enter username:" user
read -rp "Enter group:" group
create_user
;;
"Show users")
cut -d: -f1 /etc/passwd
;;
"Exit")
break
;;
*)
echo "Wrong option. Choose a number."
;;
esac
done
```

11. Результат:

```
anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ sudo ./script_addgroup_addusers_case.sh
1) Add user
2) Show users
3) Exit
#? 1
Enter username: kirill
Enter group: devopser
mkdir: created directory '/home/anastasia/devopser'
User kirill in group devopser created successfully.
#? 3

anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ tail -1 /etc/group
devopser:x:1007:

anastasia@DESKTOP-ADT12D6:~/Zeti/homework14$ tail -1 /etc/passwd
kirill:x:1010:1007::/home/anastasia/devopser/kirill:/bin/sh
```